

Contents

1	Classical ML	2
1.1	Linear Regression	2
1.2	Logistic Regression	2
1.3	Perceptron	4
1.4	Support Vector Machines	4
1.5	Principal Component Analysis	4
1.6	K-Means Clustering	5
2	Deep Learning	5
2.1	Neural Networks	6
2.2	CNNs	7
2.3	RNNs	8
2.4	Transformers	10
2.5	State Space Models	10

1 Classical ML

1.1 Linear Regression

Linear Regression is one of the most fundamental algorithms in machine learning. It is used to model the relationship between a dependent variable and one or more independent variables. The goal of linear regression is to find the best-fitting line (or hyperplane in higher dimensions) that minimizes the sum of squared differences between the observed values and the predicted values.

A simple example is the line of best fit, $\hat{y}_i = mx_i + b$. If say, we wanted to represent how much the prediction deviates from the true value y_i , we can consider minimizing the magnitude of the vector

$$\begin{bmatrix} \hat{y}_1 - y_1 \\ \hat{y}_2 - y_2 \\ \vdots \\ \hat{y}_n - y_n \end{bmatrix}$$

In that case, we've used the squared error as the loss function. Notice that the predicted value $\hat{y}_i$ is just the dot product between the vectors $\mathbf{w} = (m, b)$ and $\mathbf{x}_i = (x_i, 1)$. Then we can represent this as a matrix equation

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$

or $\mathbf{M}\beta = \hat{\mathbf{y}}$. The goal then is to minimize $\mathbf{M}\beta - \mathbf{y}$ which actually has a closed form solution

$$\beta = (\mathbf{M}^T \mathbf{M})^{-1} \mathbf{M}^T \mathbf{y}$$

Linear regression does not imply linear relation. The $\mathbf{M}$ matrix can have any term in it like

$$\begin{bmatrix} \cos(x_1) & e^{x_1} & x_1 \\ \cos(x_2) & e^{x_2} & x_2 \\ \vdots & \vdots & \vdots \\ \cos(x_n) & e^{x_n} & x_n \end{bmatrix}$$

Having products of features is also valid, however one must actually realize these features were relevant to begin with. If we didn't know, for example, our data followed an exponential distribution, we might not have thought to include e^{x_i} as a feature. This works best on data that is ideally uncorrelated across features, and we strongly know in advance the form of the relation.

For example, suppose we wanted to predict property prices. We could infer the relation between land area and price should be polynomial, and that there might be an additional bias as a result of the location. The data here is low dimensional, and we can realistically infer the direct relationship between features and price.

1.2 Logistic Regression

Logistic Regression is the most basic classification algorithm. Assuming the data follows a bernoulli distribution, then for any given point of data $\mathbf{x}_n$ we want to predict the probability it belongs

to class C . So we'd want the model to output the probability $y_n = p(C|\mathbf{x}_n)$. For a Bernoulli distribution, this probability is actually equal to

$$y_n = \frac{1}{1 + e^{-\eta}}$$

where η is also called the log odds $\eta = \log\left(\frac{y_n}{1-y_n}\right)$. We want the log odds to be linear in our parameters for a couple reasons. One of which is to aid in interpretability. The equation for the output is also called the sigmoid function, and it has several nice properties in that its differentiable and continuous everywhere.

Consider the probability our weights generated the data $p(\mathbf{t}|\mathbf{w})$. For a Bernoulli distribution this "likelihood" is exponential

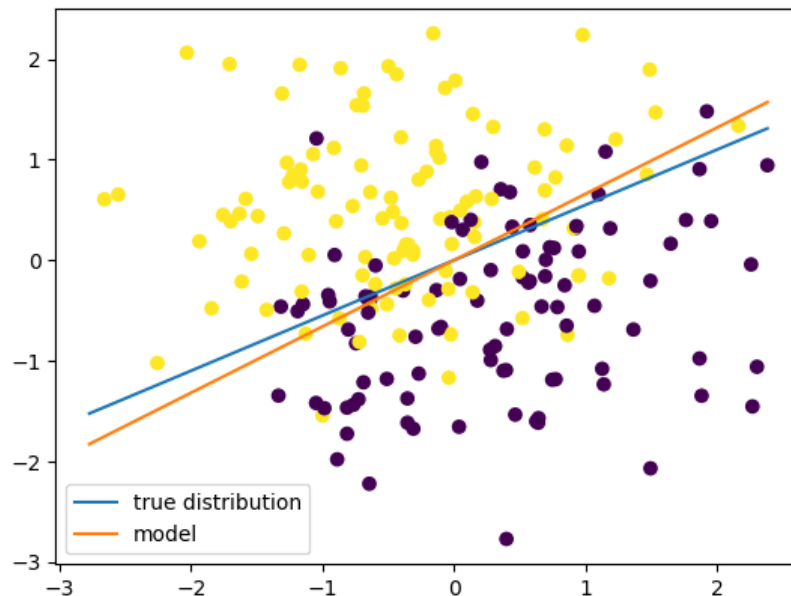
$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N [\exp(t_n \log(y_n) + (1 - t_n) \log(1 - y_n))]$$

A more friendly way to write this is by taking the log of both sides which gives us a summation

$$E(\mathbf{w}) = -\log(p(\mathbf{t}|\mathbf{w})) = \sum_{n=1}^N [t_n \log(y_n) + (1 - t_n) \log(1 - y_n)]$$

which is the cross entropy loss for binary classification. If you are familiar with cross entropy in information theory, this looks a little different than the traditional equation but it's because we already summed over all possible classes (just 2 in this case). For more clarification, see [here](#).

Note that the logarithm is a monotonic function, so minimizing the negative logarithm is equivalent to maximizing the probability. Unlike linear regression, we must use gradient descent to optimize this because the presence of the sigmoid means there is no closed form solution.



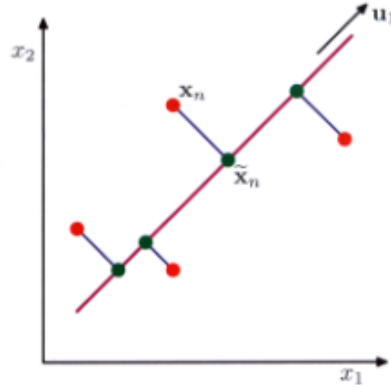
1.3 Perceptron

1.4 Support Vector Machines

1.5 Principal Component Analysis

Many classical models suffer from the curse of dimensionality. What this means is that as the number of dimensions grow, the feature space becomes sparse in data. This means we need exponentially more data as the number of dimensions grow. However, data tends to lie on lower dimensional manifolds. The vast majority of points outside of the manifold are meaningless.

Figure 12.2 Principal component analysis seeks a space of lower dimensionality, known as the principal subspace and denoted by the magenta line, such that the orthogonal projection of the data points (red dots) onto this subspace maximizes the variance of the projected points (green dots). An alternative definition of PCA is based on minimizing the sum-of-squares of the projection errors, indicated by the blue lines.



Principal component analysis aims to reduce the dimensionality of data while retaining as much information as possible. The idea is that there are many "dimensions" that are not necessary. In order to keep the maximum information, we aim to maximize the variance of the projected points. Suppose we wish to find the one dimension that is the most important. This means finding the single unit vector $\mathbf{u}$ that maximizes the variance. The variance comes out cleanly as

$$\mathbf{u}^T \mathbf{S} \mathbf{u}$$

where $\mathbf{S}$ is the covariance matrix of the data. To maximize this value we can use lagrange multipliers to maximize

$$\mathbf{u}^T \mathbf{S} \mathbf{u} - \lambda(\mathbf{u}^T \mathbf{u} - 1)$$

where we used the fact that $\mathbf{u}$ is a unit vector to get the constraint $\mathbf{u}^T \mathbf{u} = 1$. Setting the derivative to 0, we have a stationary point at $\mathbf{S} \mathbf{u} = \lambda \mathbf{u}$, and rearranging gives us $\mathbf{u}^T \mathbf{S} \mathbf{u} = \lambda$. So the variance is maximized when $\mathbf{u}$ is eigenvector of $\mathbf{S}$ with the largest eigenvalue. Additionally, because the covariance matrix is symmetric and positive, all eigenvalues are positive and we are guaranteed to have D eigenvectors where D is the number of dimensions.

Suppose we have a dataset of dimension D and we want to reduce it to K dimensions. Then we simply take the K largest eigenvectors, and then project the data onto the subspace spanned by those vectors

$$\mathbf{Z} = \mathbf{X} \mathbf{W}$$

where $\mathbf{W}$ is the matrix of the K eigenvectors, $\mathbf{X}$ is the original data matrix and $\mathbf{Z}$ is the projected data. To properly determine the value of K , a good heuristic to use is the explained variance ratio

$$\frac{\sum_{i=1}^K \lambda_i}{\sum_{j=1}^D \lambda_j}$$

if a low value of K can explain a large percentage of the variance, then perhaps that value of K suffices. One can even plot the value of the ratio vs k and use the elbow method to determine the best value.

While PCA is powerful, it comes with significant computational complexity. Computing the covariance matrix takes $O(ND^2)$ time, and computing the eigenvectors takes $O(D^3)$ time. For large datasets, this can be unweildy, though approximation methods such as randomized SVD can significantly reduce time complexity to $O(np \log(K))$. The space complexity is also an issue because the covariance matrix scales with D^2 , but this can also be worked around using mini batching. PCA might not be the best option for lower dimensional datasets because you risk losing a lot of variance/information when you reduce the dimensions.

1.6 K-Means Clustering

2 Deep Learning

The need for more expressive machine learning models at the cost of interpretability underlies the development of deep learning models. Classical algorithms have high interpretability, rigorous error bounds, and are computationally efficient. But as seen above, we must diligently select features and have a strong understanding of the underlying behavior in the data to effectively use them, which can be a hopelessly complicated task in the case of, for example, language processing.

Very loosely speaking, we can quantify the "expressiveness" of a model by the number of points it can "shatter." This means that given a set of points, the model can be trained to perfectly classify them in any way. For example, a linear model can shatter 3 points. Given any 3 points we can find a line that perfectly classifies them. Many classical algorithms have low or finite shattering

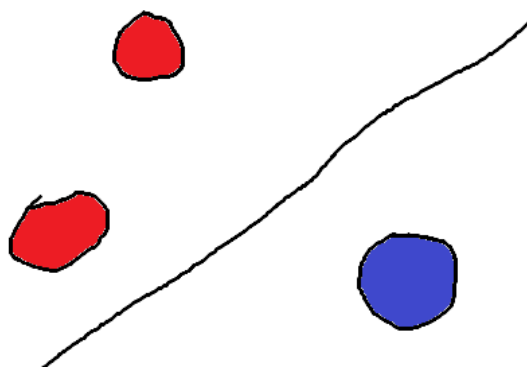


Figure 1: A linear model can only shatter 3 points.

capacity. This is very good not just for interpretability, but because we can actually put rigorous error bounds on how the model generalizes to unseen test data using VC Generalization Bounds. For very complicated tasks however, we would want more expressiveness but this usually entails a model VC dimension approaching infinity (finite shattering capacity is actually very restrictive), which means we lose PAC Learning and generalization bounds.

2.1 Neural Networks

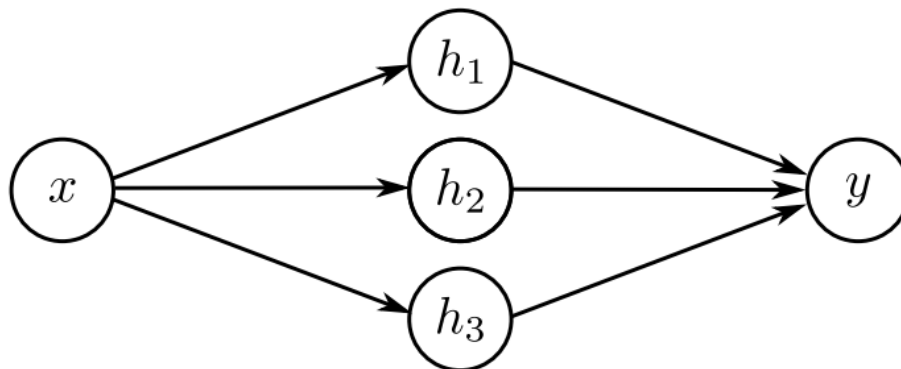


Figure 2: Shallow Neural Network.

The basic unit of a neural network is a perceptron. Each perceptron is equipped with a weight vector $\mathbf{w}$ and a bias b . An input vector $\mathbf{x}$ is fed into the perceptron, which computes the dot product $\mathbf{w} \cdot \mathbf{x} + b$. Finally, the perceptron passes this through an activation function before sending the output further through the network. The activation function is incredibly important because it allows the network to learn nonlinear behavior. If instead we just sent the output, we would simply have recreated linear regression! A common choice of activation is the rectified linear unit

$$\text{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

A famous result is the universal approximation theorem. ANY continuous function can be approximated to any precision using only a shallow neural network (only 1 layer). So theoretically a neural network with 1 layer is enough for any task.

However, for complex tasks, the number of hidden units in the single layer gets prohibitively large. It's actually better to instead of increasing the number of units, to construct more layers. Expressivity grows polynomially with hidden units per layer but exponentially with the number of hidden layers (careful, this measure of expressivity is not the same as VC dimension, but the number of regions the network can partition in input space).

Pure neural networks have largely been replaced by later architectures. Many more advanced models are actually just subsets of these neural networks but with specific structures more tailored to the task. Neural networks can still be used for data that is very high dimensional and unstructured. For example, a recommender system trying to predict products its users might buy might use a neural network. The data is high dimensional (each product, user data, time of day etc) and also unstructured.

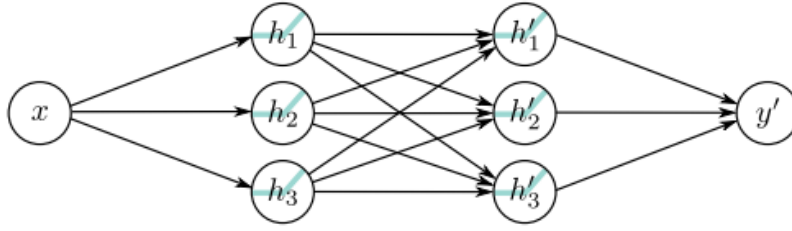


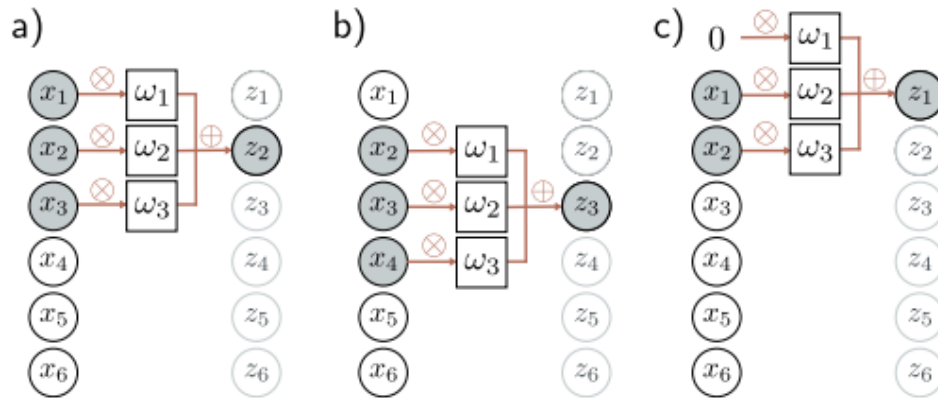
Figure 4.4 Neural network with one input, one output, and two hidden layers, each containing three hidden units.

2.2 CNNs

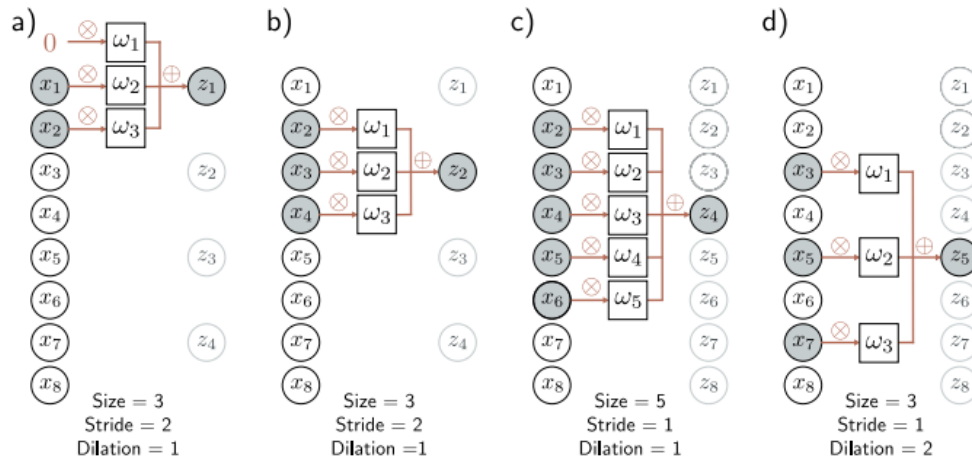
Convolutional neural networks were originally designed to classify image data, so many of the design choices were influenced by symmetries and structure of image data.

Firstly, the model's interpretation of an image should be invariant under simple transformations such as rotation or translation. For example, an image of a cat should still be classified as a cat even if we shift or rotate it slightly. Secondly, image data is extremely high dimensional. A typical image with 224×224 RGB values has over 100,000 dimensions. A shallow network with only 2 layers would require over $100,000^2 = 10$ billion parameters. Thirdly, nearby pixels are highly related to each other. If we have a pixel of a cat's nose, the surrounding pixels are highly likely to be very similar.

Using the idea that data should lie on a lower dimensional manifold, we have motivation to reduce the expressiveness (number of parameters) of our model while also taking into account the symmetries of image data.



Above is a schematic for how the layers in a CNN work. For each element z_i in the second layer, it only receives from x_{i-1} , x_i , and x_{i+1} . All elements reuse the same weights w_1, w_2, w_3 . If for example, the 1D image is translated by a certain amount, then the output will be the same result just translated by the same amount. We can further tune the interactions between layers by adjusting the stride, kernel size and dilation:



2.3 RNNs

A recurrent neural network is designed to handle sequential data. The idea is that instead of giving it all the data at once, we feed it one element at a time. Imagine our input is the sentence

"I like football."

In a traditional feed forward neural network, we might train a specific model built to process inputs of sentences of that are 4 words long (and predict whatever, it doesn't matter at the moment). Then each word in the sentence; "I", "like," ... is a separate vector to be input in the neural network with its own sets of weights and biases. However this design is poor for tasks like this for two reasons. First, sentences are usually of variable length, meaning this architecture is not able to generalize. You would have to train a new neural network for every possible sentence length.

Second, there is not enough distinction between say the 9th word in a sentence and a 10th word in a sentence to warrant a separate set of weights and biases to train for them, even less between the 29th and 30th word in a sentence. In fact, there is much statistical information that can be shared between these two words. Thus, it might make more sense to reuse the same weights and bias for each word in the sentence. Take the sentence:

"Football is a fun sport."

The word "football" should have the same meaning whether its the first or last position. Having to make the first and last inputs both learn the meaning of the word "football" is a waste of information that could've been shared.

An example RNN is shown below. At each instance in time i , this guy takes in an input vector $\mathbf{x}_i$ and then spits out an output o_i . For example, you could imagine this as a device to predict the weather. Given the current temperature, humidity etc, predict whether its sunny, rainy, etc. A model that is trained on all the data at one time might learn to hardcode the weather, associating it with time dependence instead of actually using the features. An rnn takes this into account, and is forced to learn from the current "state" $\mathbf{h}_i$ and current features $\mathbf{x}_i$.

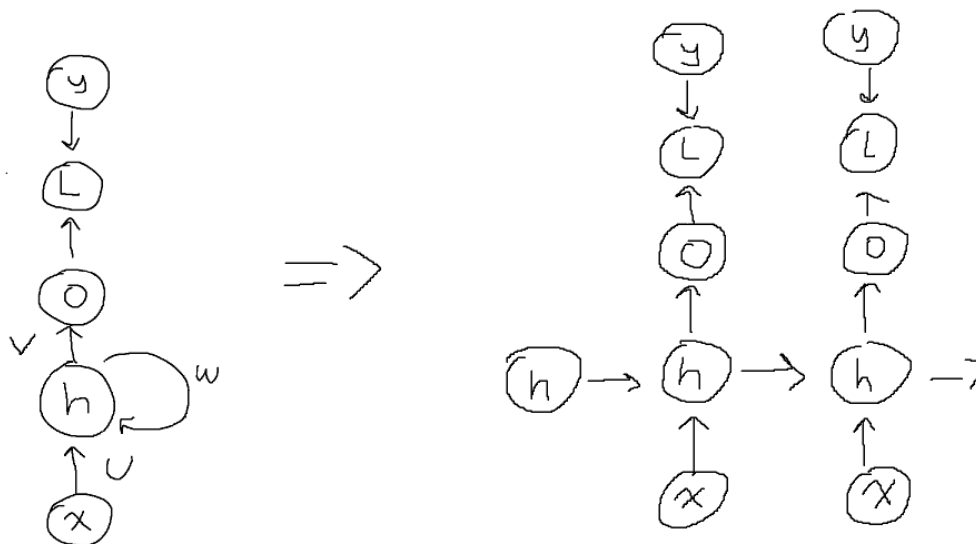


Figure 3: Example RNN Architecture.

Additionally, notice that the same weights and biases, the transformation $\mathbf{V}, \mathbf{W}, \mathbf{U}$ are being reused every time. This is in line with our motivation because how we interpret for example, weather from humidity and temperature should not depend on the day it is. In this way, our RNN is forced to learn the features instead of the pattern in the training data.

Aside: this is not the only architecture that exists, one could continuously accumulate the hidden states and only return an output at the very end (which would be more applicable to our situation).

You might have heard of "GRU" or "LSTM" before. These introduce the idea of **gating** to solve the vanishing gradient problem. Over long sequences, the gradient shrinks to 0 making training extremely slow. The idea then is to introduce gates that controls how much information the model should keep from previous timesteps. LSTMs were the first to introduce gating, with the forget, output, and input gates. The forget gate is most important, as it rewrites the current hidden state to drop past information that is no longer relevant. The gate is just a sigmoid

$$f = \sigma(\mathbf{W}_f[h_t, x_t])$$

which then multiplies the current hidden state to drop information

$$h_t = f \odot h_t$$

The gated recurrent unit (GRU) uses only two gates, the reset and update gate. After the hidden layer is updated first, the update gate determines how much of h_t to keep while the reset gate determines how much of the previous state to use.

RNNs are obviously used for sequential data. Parameter sharing means they are data efficient, so they work well on smaller datasets, as there are a small number of parameters. Additionally,

they are computationally efficient at runtime because they only have to process the current input. However, when datasets get large, training times blow up quickly because of the aforementioned vanishing gradient problem. Even if the gradient decays by a small amount say, 0.99 due to gating, this compounds over long sequences. Over 10,000 steps this becomes $0.99^{10,000} \approx 0.000043$. Additionally, RNNs must be trained sequentially, which cannot be parallelized using GPUs. This is very slow and a lot of hardware just sits around doing nothing. Finally, because RNNs use very few parameters, it becomes difficult for them to fully capture the entire history over long sequences.

2.4 Transformers

2.5 State Space Models